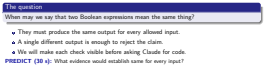
Boolean Equivalence and Finite-State Models

A confident claim still needs every check

COMSC 2043 • Discrete Structures and Critical Thinking

Week 14, Tuesday, November 10, 2026

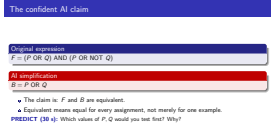
Tuesday, 75 minutes. Today we ask whether two claims really agree on every possible input, then carry that habit into a small state model. The sentence to land: one passing example is evidence, not a universal proof.



Let students answer before naming the truth table. Expected: try every input, or give a proof that covers every input. Today the small number of variables makes every row practical.



Use the smoke-alarm analogy only to make the logic concrete. Ask: if one room has smoke, what can we conclude? The whole-building claim is false. If one row differs, the equivalence claim is false.

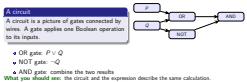


Read the AI claim with confidence, then pause. Do not reveal the counterexample yet. Ask for a test plan. Expected: all four combinations, especially a row where NOT Q changes the second term.

Words before symbols

Boolean variable	A named value that is either True or False, such as P.
Assignment	The values chosen for the variables, such as $P = False$, $Q = True$.
AND	True only when both inputs are True.
OR	True when at least one input is True.
NOT	Flips True to False, or False to True.
Expression	A rule built from variables and Boolean operations.
Equivalence	Two expressions have equal outputs for every assignment.

Define each term as it first appears. Keep the words connected to the two expressions on the previous slide. Ask students to translate one symbol into a sentence.



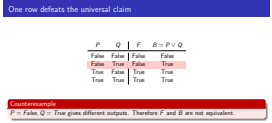
The drawing is a circuit interpretation, not a physical hardware claim. Point to each wire and say which subexpression it carries. The point is that checking a formula can also check a circuit diagram.

A truth table checks all assignments

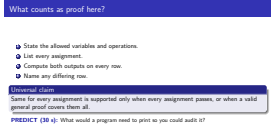
P	Q	$P \vee Q$	$P \vee \neg Q$	F
False	False	False	True	False
False	True	True	False	False
True	False	True	True	True
True	True	True	True	True

Truth table: a table that lists an expression's output for every assignment.
What you should see: the second row already makes the AI claim suspicious.

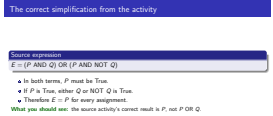
Fill this table slowly. Ask students to compute each intermediate column, not jump to the final answer. A truth table is exhaustive here because two variables have only four assignments.



Box the only mismatch. Say the conclusion in both directions: the claim fails, and we know exactly why. This is the central false AI claim for the deck.



This is the course method in miniature. Ask for visible intermediate work and a named failure, not only a final Boolean. This prepares the prompts.



Connect to the existing source activity without rewriting it. The algebraic idea is P is required, and Q has both possibilities. Then verify with the same four rows.



- two AND gates
- one NOT gate
- one OR gate

PREDICT (30 s): Which circuit output should match P on all four rows?

Keep the visual parallel with the previous circuit. The same equivalence habit applies when an expression is drawn as gates: check the meaning, not just the shape.

Prompt 01: small steps first

PROMPT FOR CLAUDE

```
Act as a parallel checking partner for a two-variable Boolean example. Before writing code, state what an
"assignment" to Boolean variables is, no assignment, and equivalence mean in plain language. Use only the expressions
AND, OR, and NOT. Keep the example synthetic and self-contained. Then propose the smallest Python standard-
library evaluator you could need for a truth table. Do not claim two expressions are equivalent until every
assignment has been checked. Show one predicted row before showing the rest.
```

What you should see: one visible Boolean result, plus a visible failure for a missing variable if you try it.

PREDICT (30 s): What should *P* AND NOT *Q* return for *P* = *True*, *Q* = *False*?

Paste the prompt into Claude. The desired artifact is a small evaluator with four explicit node forms. Ask students to predict True before running. Recovery: compare the structure with `code/boolean_fsm.py`; do not accept a silent default for missing variables.

Prompt 02: enumerate every assignment

PROMPT FOR CLAUDE prompts/02_truth_table_counterexample.md

```
Write a small Python program using only the standard library. Define P = (P OR Q) AND (P OR NOT Q) and the
proposed simplification S = P OR Q. Evaluate both expressions for all four Boolean assignments of P and Q, in
the order (True, True), (True, False), (False, True), (False, False). Print a table with columns P, Q, P, and S,
and S, then print the exact assignments where they differ. Do not simplify the expressions by inspection. The
reported check is one mismatch, at P=False and Q=True.
```

What you should see: four rows in fixed order; the row $P = \mathit{False}$, $Q = \mathit{True}$ is the mismatch already visible on the slide.

Common mistake: A single test row cannot establish equivalence.

Have students compare Claude's printed table to slide 8. If the order differs, ask for an explicit ordering so the evidence is easy to audit.

Prompt 03a: require the exact counterexample

PROMPT FOR CLAUDE prompts/03_equivalence_pair.md

Extend a small standard-library Boolean evaluator with an equivalence checker. It must enumerate every assignment for the variables that occur and return a list of counterexamples, not just True or False. Check that `X * (Y and Z)` and `(X * Y) and (X * Z)` are equivalent, and that `X * (Y or Z)` and `(X * Y) or (X * Z)` are equivalent too. Print the counterexample for the second pair exactly as a mapping from variable names to Boolean values. State why an empty counterexample list supports equivalence only for the variables and operators actually supplied.

What you should see: equivalence → `False` and counterexample → `{"P": False, "Q": True}`.
PREDICT (30 s): Why is returning only False weaker than returning the assignment?

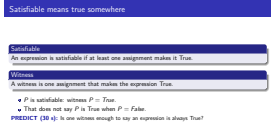
The assignment makes the failure reproducible. Ask students to compare the exact dictionary, not merely a summary. If a model returns the wrong row, rerun the visible four-row table and use the committed reference as recovery.

Three ways a check can speak

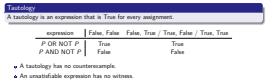
Output	Meaning	Example
matching rows	every listed assignment agrees	E matches P on all four rows
counterexample	one assignment disagrees	F vs B at False, True
witness	one assignment makes an expression True	P has witness $P = True$

What you should see: the shape of the evidence must match the strength of the claim.

This is a bridge to satisfiability. A witness is useful, but it answers a different question from equivalence.



Pause for the answer: no. This distinction is a required AI failure mode. Use the two assignments for P as a tiny counterexample to the mistaken inference.



Use everywhere and nowhere language before the vocabulary. The table is small enough to inspect, but the same distinction matters when there are many variables.

Prompt 03b: do not overclaim from a witness

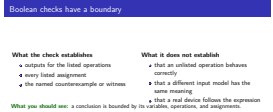
PROMPT FOR CLAUDE prompts/03_implication_satisfied.txt

Using the same evaluator, write three separate checks. First, return one satisfying witness for P, meaning one assignment that makes P true. Second, decide whether P is a tautology, meaning true for every assignment, and if it is not, return one falsifying assignment. Third, decide whether P is satisfiable, and if it is, return one witness. Explain in one sentence why a satisfying witness for P does not prove that P is always true. Do not use the words equivalent or always without showing the assignments that justify them.

What you should see: one witness for P, a counterexample to P being tautological, no counterexample for P OR NOT P, and no witness for P AND NOT P.

Common mistake: I found one True row does not mean all rows are True.

Ask students to point to the exact evidence for each claim. If Claude says the witness proves a tautology, return to the two-row table for P and ask which row it skipped.



This is the first use of model limit in the deck: the boundary of what the chosen representation and checks can establish. Carry this sentence into the state-model section.

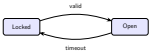
Finite-state machine

A finite-state machine (FSM) is a model with a finite list of states and rules for how named inputs move between them.

State	A named condition the model can be in, such as Locked.
Input	A named event the model receives, such as valid.
Transition	A rule mapping a current state and input to a next state.
Start state	The state where a run begins.
Trace	The start state and the sequence of transitions taken.

Define state-model terms before drawing. Emphasize that an input is an event in the model, not proof that a physical event really occurred.

A complete two-state badge model



- Start state: Locked.
- Declared inputs: valid and timeout.
- This is a synthetic teaching model, not a real access-control claim.

Read the arrows as explicit rules. The model has two states and two inputs; later we will deliberately omit one arrow to see whether validation notices.

The supplied trace



step	current	input	next
start	Locked	–	Locked
1	Locked	valid	Open
2	Open	timeout	Locked
3	Locked	valid	Open

What you should see: the trace includes the start state and every next state.

Trace the arrows with a finger. Ask for the state after each input. The final state alone is not enough evidence because different traces could end in the same place.

Prompt 04: print the trace

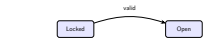
PROMPT FOR CLAUDE prompts/04_fsm_trace.md

Implement a tiny finite-state-machine runner in Python with no third-party packages. The machine has states Locked and Open, inputs valid and timeout, start state Locked, and these transitions: (Locked, valid) -> Open and (Open, timeout) -> Locked. Any other (state, input) pair leaves the state unchanged. Run the input sequence valid, timeout, valid and print the start state plus each resulting state. Also print a visible error for an input that is not in the declared alphabet. The expected state sequence is Locked, Open, Locked, Open. Do not invent authentication facts or physical-reader behavior.

What you should see: Locked, Open, Locked, Open, with the three inputs visible between them.
PREDICT (30 s): What is the state after valid, timeout, valid?

The prompt asks for explicit transitions and no implicit self-loop. Compare the printed trace to the table on slide 27. If the start state is missing, ask Claude to include it.

An incomplete model can look plausible



- States: Locked, Open. Inputs: valid, timeout.
- Present rule: Locked + valid → Open.
- Missing rule: Open + timeout → Locked.

Common mistake: Do not invent a self-loop just because no arrow is drawn.

Show that a partial model can still answer some questions. It cannot claim total behavior until every state/input pair has a rule or an explicit rejection policy.

Prompt 05: name the exact gap

PROMPT FOR CLAUDE prompt/05_model_limit.md

```
Write a validator for a finite-state machine whose declared states are Locked and Open and whose input alphabet is
  valid and timeout. The incrementally incomplete transition table contains only (Locked, valid) -> Open. List
  every missing (state, input) pair, then state in one sentence what this validator can establish and what it cannot
  — the missing transition masked by the badge trace. Do not assume an implicit self-loop counts as a declared
  transition. Explain in one sentence what this validator can establish and what it cannot establish about a real-
  access system.
```

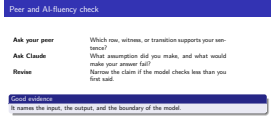
What you should see: exactly one missing pair: (Open, timeout), plus a named model limit.
PREDICT (30 s): Which state/input pair is absent?

The validator should list the pair, not merely say incomplete. Then ask for the model-limit sentence: it cannot establish authorization, a physical reader condition, or behavior of a missing state such as Denied or Expired.

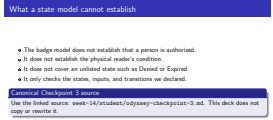
Tuesday activity: defend one claim

- Choose one expression pair or one small state trace.
- Write the rule, variables/states, and expected result before using Claude.
- Run one prompt and compare its visible output with your hand check.
- Mark one match and one possible drift.

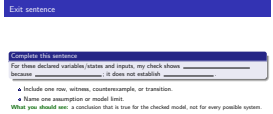
Give students individual work time. This is a graded discussion resource: peers will see the work, so use synthetic examples and do not include private data.



Circulate for claims that are stronger than the evidence. Ask students to revise rather than hide a mismatch.



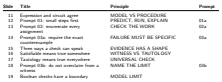
Point to the canonical path exactly. Do not paraphrase or replace the authoritative Checkpoint 3 text here. The package's later student activity will link to it.



Collect the exit sentence. A strong example names the exact counterexample and says the result is limited to the declared variables and operations. Preview Thursday's defense without adding new submission rules here.

Slide	Title	Principle	Prompt
2	Today's question	EQUIVALENCE IS A UNIVERSAL	
3	A smoke alarm for a strong claim	CHECK THE WORK	
4	The confident AI claim	PREDICT BEFORE RUNNING	
5	Words before symbols	DEFINE THE MODEL	
6	From expression to circuit	MODEL VS PROCEDURE	
7	A truth table checks all assignments	EXHAUSTIVE CHECK	
8	Why one counterexample is enough	ONE IS ENOUGH	
9	What counts as proof here?	SOURCES, RULES, WORK, CHECK	
10	The correct simplification from the activity	COMPARE CLAIMS	

Use this index to find the slide, principle, and prompt file.



Slide	Title	Principle	Prompt
11	Expression and circuit agree	MODEL VS PROCEDURE	
12	Prompt 01: small steps first	PREDICT, RUN, EXPLAIN	01a
13	Prompt 02: enumerate every assignment	CHECK THE WORK	02a
14	Prompt 03a: require the exact failure	FAILURE MUST BE SPECIFIC	03a
15	Three ways a sketch can speak	EVIDENCE HAS A SHAPE	
16	Satisfiable means true somewhere	WITNESS VS TAUTOLOGY	
17	Tautology means true everywhere	UNIVERSAL CHECK	
18	Prompt 03b: do not overclaim from a sketch	NAME THE LIMIT	03b
19	Boolean sketches have a boundary	MODEL LIMIT	

Use this index to find the slide, principle, and prompt file.

Slide	Title	Principle	Prompt
20	From Boolean outputs to states	MODEL THE NEXT STEP	
21	A complete two-state badge model	EVERY TRANSITION IS EXPLICIT	
22	The supplied trace	TRACE THE EVIDENCE	
23	Prompt 04: print the trace	PREDICT, RUN, EXPLAIN	04a
24	Prompt 05: name the most plausible	VALIDATE THE MODEL AGAINST EVIDENCE	
25	Prompt 05: name the exact gap	VALIDATE THE MODEL	05a
26	Tuesday activity: defend one claim	INDIVIDUAL CHECK	
27	Peer and AI fluency check	CRITIQUE THE EVIDENCE	
28	What a state model cannot establish	MODEL LIMIT	

Use this index to find the slide, principle, and prompt file.

Slide	Title	Principle	Prompt
29	Exit sentence	BOUNDED CONCLUSION	

Use this index to find the slide, principle, and prompt file.